Ders 2 Genişletilmiş Notu: Bilgisayar Mimarisi Tekrarı, Veri Tipleri ve IEEE 754, Tip Dönüşümü, Operatörler, printf Biçimlendirme ve enum

Ders 2 Genişletilmiş Notu: Bilgisayar Mimarisi Tekrarı, Veri Tipleri ve IEEE 754, Tip Dönüşümü, Operatörler, printf Biçimlendirme ve enum

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Bilgisayar Mimarisi Tekrarı
- 2. C Programının Yapısı ve Değişken Türleri
- 3. Tam Sayılar ve sizeof
- 4. Ondalıklı Sayılar: IEEE 754
- 5. Karakterler ve ASCII
- 6. Tip Dönüşümü (Casting)
- 7. Virgül Operatörü
- 8. Arttırma Operatörleri
- 9. İlişkisel ve Mantıksal Operatörler
- 10. printf ile Biçimlendirme
- 11. Tam Sayı Bölmesi
- 12. Atama: L-value ve R-value
- 13. Kapsamlı Örnek: Dönüşümler ve İkiye Tümleyen
- 14. printf İçinde Arttırma ve m++ * m++ Tartışması
- 15. İkili Mantık Tekrarı
- 16. m++ * m++ İfadesine Assembly ile Bakmak
- 17. Numaralandırma (enum)
- 18. Ömür (Duration): Stack ve Heap

1. Bilgisayar Mimarisi Tekrarı

Bu saat normalde uygulama saatidir. Hoca, konulardan geri kalmamak ve temel konuları tamamlamak için birkaç hafta bu saati de ders olarak kullanacağını söyledi. Geçen derste bilgisayar sistemlerinden kısaca söz edilmişti. Bu derste aynı konunun biraz daha derinine inilerek başlandı. Slaytlar ve derste yazılan kodlar, paylaşılan ders notları bağlantısında bulunuyor.

1.1 Von Neumann Mimarisi

Hoca, günümüz bilgisayar sistemlerinin çoğunun iki farklı mimariden oluştuğunu söyleyerek başladı. Altyazıda yalnız birincisi, Von Neumann mimarisi anlatılıyor.

Bu mimaride sistemde bir işlemci ve bir ana bellek bulunur. Programlara ait komutlar da veriler de aynı ana bellekte saklanır. Komutların ve verilerin tek bir bellekte tutulması, mimarinin ayırt edici özelliğidir. Bunlara ek olarak giriş/çıkış (input/output) sistemleri vardır.

İşlemcinin içinde temel olarak aritmetik-mantık birimi (ALU) bulunur. Programların komutları burada yürütülür. ALU'nun görevi yürütmedir (execution). Hoca bu terimi "bir emrin yerine getirilmesi" olarak açıkladı. Türkçede genellikle "icra etmek" denir. Komut toplama ise, toplama işleminin fiilen yapılmasıdır. Komutların teker teker çalıştırılmasına da run dediğimizi hatırlattı.

1.2 Yazmaçlar

Yazmaçlar (register), işlemcinin içinde çeşitli amaçlarla veri saklamak için kullanılan kayıt birimleridir. Hoca bunları kullanım amacını sorarak sıraladı:

- **Sıradaki komut hangisi?** Programdan kaçıncı komutun getirileceği, program sayacı (program counter) sayesinde belirlenir.
- **Gelen komut nerede saklanacak?** Bunun için bellek tampon yazmacı (memory buffer register) ve komut yazmacı (instruction register) vardır.
- **Komutlara ait veriler nerede saklanacak?** Bellekle ilgili yazmaçlarda ve AX, BX gibi genel amaçlı yazmaçlarda.

1.3 Bellek Hiyerarşisi

İşlemcinin içinden bellek tarafına geçtiğimizde farklı seviyelerde bellekler görürüz:

1. İşlemcinin içindeki yazmaçlar. Bunlar da bir bellektir.
2. Önbellek (cache memory).
3. Ana bellek (RAM). Programlarımızı ve verilerimizi sakladığımız yer burasıdır.
4. Disk yapıları: sabit disk (hard disk) ya da SSD.

Bu seviyelerde, yerine göre kalıcı, yerine göre geçici (uçucu) veri saklanır. Kapasite alt seviyelerde, hız üst seviyelerde daha fazladır. Hoca bu bilgiyi genel kültür gibi düşünebileceğimizi söyledi.

1.4 Program Nasıl Çalışır?

Von Neumann mimarisinin bu derste konuşulma nedeni, programın nasıl çalışacağını anlamaktır. Programı ve verileri ana bellekte tutarız. İşlemcideki program sayacı, bir sonraki adımda çalıştırılacak komutun bellekte nerede olduğunu söyler. Bellekte adresler vardır. "Şu adresteki komutu getir" denir.

Hoca bunu bir benzetmeyle gösterdi. Bir "hello world" programı yazdığınızda, kodun bir kısmı bir adreste, bir sonraki adreste `return 0` duruyormuş gibi düşünebilirsiniz. Program sayacı nereyi gösterirse o komut getirilir ve çalıştırılır. Bunun yalnız bir benzetme olduğunu da ekledi. Bellek elektronik bir yapıdır, 0 ve 1'leri yani voltaj ve akım değerlerini tutar. İçinde kaynak kod satırları böyle dizili durmaz.

1.5 İşletim Sistemi

Sistemi tek bir program kullansaydı olay bu kadar basit olurdu: bir program, bir ana bellek, bir işlemci. Ama aynı anda birden fazla program çalışır. Hocanın örneği kendisiydi: O konuşurken bir yandan yazıyor, bir yandan başka programlar çalışıyor.

Bu programları işletim sistemi idare eder. İşlemciyi, belleği ve giriş/çıkış kaynaklarını, kısaca donanımı, programlar arasında zaman paylaşımı esasına ya da belli bir zamanlama (schedule) esasına göre paylaştırır. Yönetir, kontrol eder, koordine eder. Hoca bunu bir yönetici yazılım olarak düşünebileceğimizi söyledi.

2. C Programının Yapısı ve Değişken Türleri

Her C programı bir `main` fonksiyonu içermek zorundadır. İhtiyacımız olan kütüphaneleri `#include` ifadesiyle ekleriz ve burada başlık dosyalarını (header file) kullanırız. Hoca başlık dosyalarının nasıl yazılacağının ileride konuşulacağını söyledi. Yorum satırları gibi bilinen konuları hızlı geçti.

> **Ders sonrası not:** Hoca başlık dosyalarını "bağlı olduğu fonksiyonların gövdelerini çağırmakla yükümlü özel dosyalar" diye tanımladı. Doğrusu büyük olasılıkla şudur: Başlık dosyaları fonksiyonların gövdelerini değil bildirimlerini içerir. Gövdeler derlenmiş kütüphanelerdedir ve bağlama (linking) aşamasında programa eklenir.

2.1 Dört Temel Tür

Hocaya göre C'de temelde dört değişken türü vardır: tam sayı, tek duyarlıklı (single precision) ondalıklı sayı, çift duyarlıklı (double precision) ondalıklı sayı ve karakter. Diğerleri bunların türevleridir. Tam sayı dizisi, tam sayılardan oluşan bir yapıdır. String doğrudan yoktur, karakter dizisi olarak ifade edilir. Boolean diye bir tür de yoktur. Onun yerine bir tam sayıyı 0 ya da 1 olarak kullanırız.

2.2 Bildirim ve Tanım

Hoca iki kavramı ayırdı. Bir değişkene değer atamadan yalnız etiketini söylemeye bildirim (declaration) dedi. Hem bildirim hem de değer ataması yapılan duruma ise tanım (definition) dedi. Gerekçesi, değer atanmış değişkenin bellekte sabit bir yerde durması gerektiğiydi. Yalnız bildirilen değişkene ilk açılışta belki doğrudan yer ayrılmayabileceğini, ilk kullanıldığında ayrılacağını söyledi.

> **Ders sonrası not:** Hoca, değer atanmayan `int x;` satırının yalnız bildirim olduğunu söyledi; doğrusu büyük olasılıkla şudur: Fonksiyon içindeki `int x;` hem bildirim hem tanımdır, bellek ayrılır ama başlangıç değeri yoktur. Yalnız bildirim örnekleri `extern int x;` ve fonksiyon prototipidir. `int x = 5;` ise ilk değer verilmiş tanımdır.

3. Tam Sayılar ve sizeof

3.1 signed ve unsigned

Tam sayıların değer aralığı önüne yazılan `signed` ve `unsigned` sözcükleriyle değişir. Hiçbir şey yazmazsanız `int`, negatif sayıları da kapsadığı için `signed` olur. Slaytta `int x = -12;`, `unsigned int x = 5;` ve `short (int) x = 2;` örnekleri vardı.

`int` 32 bittir. `signed` durumda 1 bit işarete gider, kalan 31 bit değeri saklar. Aralık, negatif tarafta -2^{31}'den -1'e, sonra 0'dan $2^{31}-1$'e kadardır.

`unsigned` durumda işaret derdi yoktur. Bit dizisi hepsi 0 olan değerden hepsi 1 olan değere kadar gider. Ulaşılan en büyük değer $2^{32}-1$'dir. Hoca, sonucun tek sayı olacağını da belirtti.

Slaytta aralıklar "0 ile 2^{32}" ve "-2^{31} ile 2^{31}" diye yazılıydı. Hoca üst sınırlardaki -1 eksikliğini derste gösterip el yazısıyla düzeltti. Değerler python ile hesaplandı: $2^{31}-1 = 2147483647$, $2^{32}-1 = 4294967295$.

3.2 short, long, long long

Slayttaki boyut bilgisi şöyleydi:

- `short int`: en az 16 bit
- `long int`: en az 32 bit
- `long long int`: en az 64 bit
- `size(short)` $\leq$ `size(int)` $\leq$ `size(long)`

Soru: 32 bitlik bir `int` varken neden 16 bite indiriyoruz? Büyük olsa, ileride de kullansak işimize yaramaz mı? **Cevap:** Öğrenciler "daha az yer kaplar", "gömülü sistemlerde veri tasarrufu yaparız" diye cevap verdi. Hoca gömülü sistemlerdeki ve eski bilgisayarlardaki bellek kısıtını açtı.

Hocanın karşılaştırması şuydu. Bugünkü bilgisayarlarda 8, 12, 16 GB RAM görürüz. Çok daha büyüklerini destekleyen sistemler de var. Ama 96 GB bile çok fazla değildir: her biri 4 GB'lık video dosyası düşünürseniz en fazla 20 tane film saklarsınız. Gömülü sistem ve eski bilgisayar seviyesinde ise yeteri kadar bellek yoktur.

> **Ders sonrası not:** Hoca 96 GB için "en fazla 20 film" dedi; python ile hesaplandı, $96/4 = 24$ film eder ($20 \times 4 = 80$ GB). Hocanın amacı büyüklük hissi vermekti, sonuç değişmiyor.

Yeteri kadar bellek yoksa diğer programların çalışması engellenir. İşletim sistemi RAM'i size ve diğer programlara tahsis eder. Siz çok fazla bellek harcarsanız bellek ihtiyacı olan diğer süreçlere (process) ve yerine göre kendinize sıkıntı çıkarırsınız. Bu yüzden gerektiğinde daha az, gerektiğinde daha çok bellek kullanmak işe yarar.

3.3 sizeof Bir Operatördür

`sizeof` yazılışıyla fonksiyona benzer, ama bir operatördür. Hoca, geçen hafta verdiği operatör tablosunu açıp tip dönüşümü operatörünün yanında `sizeof` operatörünü gösterdi. Sonra derleyicide denedi: `sizeof(int)`, `sizeof(short)`, ve `long` ile deneme yaptı.

İlk çıktı 4, 2, 4 oldu. Hoca "demek ki `long long` yazmak gerekiyor" dedi ve o zaman 4, 2, 8 elde etti. Sonuçlar bayt cinsindendir. Sonraki derslerde `sizeof` özellikle `malloc` konusunda kullanılacak.

> **Ders sonrası not:** Hocanın ekranında `long` 4 bayt çıktı. Bu not hazırlanırken Linux üzerinde x86-64 için gcc ile denendi, `sizeof(long)` 8 çıktı. `long` boyutu derleyiciye ve sisteme göre değişiyor, sabit tek değer değil. Slayttaki "en az 32 bit" ifadesi de bununla uyumlu.

4. Ondalıklı Sayılar: IEEE 754

Ondalıklı sayılar, üstel gösterim (exponential notation) denen bir biçimde saklanır. Bunun standardı IEEE 754'tür. Hoca arama yapınca çok sayıda örnek sayfa çıkacağını söyledi ve bir üniversite sayfası açtı. Aradığımız her sayfaya bakmamamızı, en azından bir üniversite sayfası olup olmadığına dikkat etmemizi önerdi. Single precision 32 bit, double precision 64 bittir.

Bir sayıyı 32 bite sığdıracaksanız virgülden önceki tarafı bir yere, virgülden sonraki tarafı başka bir yere koymanız gerekir. 32 bitin tamamı tek bir kısma ayrılamaz. Bu gösterimle yaklaşık -10^{38} ile 10^{38} arasında sayılar saklanabilir. Hoca bu sınırların yaklaşık olduğunu söyledi.

`sizeof` bir boyut döndürdüğü için, tipi `float` da olsa `double` da olsa sonucu bayt cinsinden tam sayıdır ve tam sayı olarak yazdırılır. Derste `sizeof(float)` 4 ve `sizeof(double)` 8 görüldü.

4.1 Sayının Parçaları

Hocanın anlattığı mantık şöyleydi:

1. Sayı artı ise işaret biti 0, eksi ise 1 olur.
2. Sayı, virgülden önce tam kısımda tek basamak kalacak biçimde yazılır. Geri kalan basamaklar virgülden sonrasına geçer. Hocanın sözlü örneğinde virgülden sonra 278 gibi basamaklar vardı.
3. Sayı, bir taban kuvvetiyle çarpılmış olarak ifade edilir. Bu kuvvet üs (exponent) alanına gider.
4. Virgülden sonraki kısım kesir (fraction) alanına gider. Bu alana mantis de denir.

Alan boyutları şöyledir:

Biçim	Toplam	İşaret	Üs	Kesir
Tek duyarlık (`float`)	32 bit	1	8	23
Çift duyarlık (`double`)	64 bit	1	11	52

> **Ders sonrası not:** Hoca sayının "10 üzeri bilmem kaç" olarak yazıldığını söyledi; doğrusu büyük olasılıkla 2'nin kuvvetidir, çünkü sayı ikilik tabanda normalize edilir. Aşağıdaki iki örneğin bit dizileri bu kabulle python ile hesaplandı ve `float` paketlemesinin çıktısıyla aynı çıktı.

4.2 Örnek: 0.15625

Slaytta 0.15625 değerinin saklanması gösterildi. Hoca adımları sözlü olarak geçti. Ara adımlar aşağıda eklendi ve python ile hesaplandı (ara adım eklendi).

1. Verilen: 0.15625, pozitif, işaret biti 0.
2. İkilik yazım: $0.15625 = 1.01_2 \times 2^{-3} = 1.25 \times 2^{-3}$.
3. Üs alanı: $-3 + 127 = 124$, yani `01111100`.
4. Kesir alanı: virgülden sonraki `01`, sağına 21 tane sıfır gelir.

```
işaret  üs        kesir
0       01111100  01000000000000000000000
```

4.3 Örnek: 25.68

Hoca bir IEEE 754 hesaplayıcı sayfası açıp 25.68 girdi. Sayı pozitif olduğu için işaret biti 0 çıktı. Hoca sonucun iki parçaya bölündüğünü, 32 bitlik mi 64 bitlik mi olduğundan tam emin olmadığını söyledi.

Tek duyarlık için adımlar şöyledir (ara adım eklendi, python ile hesaplandı):

1. Verilen: 25.68, pozitif, işaret biti 0.
2. Normalize yazım: $25.68 = 1.605 \times 2^4$.
3. Üs alanı: $4 + 127 = 131$, yani `10000011`.
4. Kesir alanı: 0.605'in 23 bitlik karşılığı.

```
işaret  üs        kesir
0       10000011  10011010111000010100100
```

Bu bit dizisinin geri okunan değeri 25.68000030517578'dir (python ile hesaplandı). Yani 25.68 tek duyarlıkta tam olarak saklanamıyor, en yakın değer saklanıyor.

Hoca bu hesaplayıcılarla deneme yanılma yaparak iki yönü de görebileceğimizi söyledi: virgüllü sayının ikilik gösterimi ve ikilik bir dizinin geri virgüllü sayıya çevrilmesi.

> **Ek bilgi (derste anlatılmadı):** Yukarıdaki üs alanına gerçek üs değil, üstüne sabit bir sayı eklenmiş değer yazılır: tek duyarlıkta 127, çift duyarlıkta 1023. Normalize edilmiş sayının virgülden önceki 1'i de saklanmaz, bu yüzden kesir alanı yalnız virgülden sonrasını içerir. Bu iki kural, 0.15625 ve 25.68 için python ile hesaplanan bit dizileriyle tutarlıdır.

5. Karakterler ve ASCII

Karakterler günlük hayatta kullandığımız metinleri oluşturan sembollerdir ve tek baytlık gösterimle (single byte representation) saklanır. Bir bayt, 0 ile 255 arasında 256 farklı sembol demektir. Bu sembolleri ASCII tablosunda görebiliriz.

Hoca komut satırında Alt tuşuyla sayı girerek bunu gösterdi. Alt ile birlikte girilen sayı 0-255 arasında olmalıdır:

Alt + sayı	Karakter
124	| (düz çizgi)
65	`A`
48	`0`
49	`1`

Bu dört eşleşme python ile hesaplandı ve doğru çıktı. Tablonun devamında genişletilmiş (extended) kısımlar da vardır. C'de string yoktur, onun yerine karakter dizisi vardır ve bunu kendimiz yönetmek zorundayız.

> **Ek bilgi (derste anlatılmadı):** Standart ASCII büyük olasılıkla 7 bittir (0-127). Hocanın "extended" dediği 128-255 aralığı da büyük olasılıkla kod sayfasına göre değişir. Bu yüzden Alt ile girilen yüksek sayılar her sistemde aynı sembolü vermeyebilir.

6. Tip Dönüşümü (Casting)

Buraya kadar değişkenler hatırlandı. Hoca dersin asıl konusunun buradan sonrası olduğunu söyledi.

Cast sözcüğü, bir kalıba demir dökmek gibi bir şeyi anlatır. Hocanın benzetmesi buydu: Bir veriyi başka bir veri tipinin kalıbına döküyorsunuz. Programlamada bir değişkenin değerini başka bir tipe dönüştürmek için kullanılır.

Küçük tipte bir veriyi büyük tipe ya da büyük tipte bir veriyi küçük tipe dönüştürmek her zaman istediğimiz bir şey değildir. Uygulamaya göre iki yön de gerekir, ama bunu bilinçli yapmak gerekir. Derleyici sizi her zaman uyarmaz. Bazı dillerde uyarı alınabilir. İşlemin yazımı basittir: Parantez içine hedef tipi yazarsınız.

İki türü vardır: gizli olan örtük (implicit) ve açıktan yazılan açık (explicit) dönüşüm.

Soru: Bu ikisinden hangisi daha tehlikeli, hangisi gözden daha kolay kaçar? **Cevap:** Örtük olan. Çünkü ne olacağını kodun okunuşundan göremeyebilirsiniz.

6.1 Örtük Dönüşüm

Bir satırda iki sayının toplandığını görürsünüz, gözünüze takılan bir şey olmaz. Ama kodun yukarısında bir yerde o sayılardan biri `float` değil de `int` tanımlanmış olabilir. Hoca bunu denedi:

```
int x = 1;
float f = 2.3;
float y = x + f;
```

Derleyici ne hata verdi ne uyarı. Hoca, uyarı verse belki ayılacağımızı söyledi. Sonra `float` yerine `double` yazdı, yine ses çıkmadı. Burada `x`, farkında olmadan 1'den 1.0'a dönüşür. Buna örtük dönüşüm denir.

6.2 Açık Dönüşüm

Aynı işlem açık dönüşümle şöyle yazılır:

```
double z = (double)x + (double)f;
```

Soru: Bu satırda kaç operatör çalışıyor, hangileri? **Cevap:** Öğrenciler eşittir ve artıyı saydı. Hoca iki `(double)`'ı da ekledi. Bunlar da fonksiyon değil, tip dönüşümü operatörüdür.

Sıralamayı operatör öncelik tablosu belirler. Atama tablonun en altındadır. Toplama daha yukarıdadır. Cast ise toplamadan da yukarıdadır. Yani önce dönüşümler, sonra toplama, en son atama yapılır.

> **Ders sonrası not:** Hoca, tablodaki cast satırının sağdan sola (right to left) olmasına bakarak önce sağdaki `(double)f`, sonra soldaki `(double)x` yapılacak dedi; doğrusu büyük olasılıkla şudur: Birleşme yönü `(double)(int)x` gibi iç içe yazımda hangi cast'in önce uygulanacağını söyler. Toplamanın iki yanındaki ifadelerin hangi sırada hesaplanacağını belirlemez.

6.3 Yanlış Biçim Belirteci

Hoca bu kodu yazdırırken `%d` kullandı: `printf("%d\n", z);`. Derleyici hata vermedi, ama ekranda alakasız büyük bir sayı çıktı. `%d` tam sayılar için kullanılan bir belirteçtir, `z` ise `double`'dır. Belirteci `%lf` yapınca beklenen 3.3 göründü. Hoca bunu "3.3'ün ikilik gösterimi tam sayıya dönüştürülmeye çalışılıyor, o yüzden büyük bir sayı görüyoruz" diye açıkladı.

> **Ders sonrası not:** Hoca "64 bitlik şey 4 bitlik integer" dedi; doğrusu 4 bayt, yani 32 bittir. Hocanın kendisi de birkaç dakika sonra `int` için 4 bayt dedi. Ayrıca buna "örtük dönüşüm" dedi; doğrusu büyük olasılıkla burada hiçbir dönüşüm yapılmaz, biçim belirteci yanlıştır ve sonuç tanımsızdır. gcc ile denendi: `-Wall` şu uyarıyı verdi: `format '%d' expects argument of type 'int', but argument 2 has type 'double'`. Aynı kod bu makinede `-O0` ile -2124185327, `-O2` ile 0 yazdı. Yani görülen sayıya güvenilmez.

6.4 Küçükten Büyüğe, Büyükten Küçüğe

Açık dönüşümde de dikkat gerekir. `char` 1 bayt, yani 8 bittir. `int` 4 bayttır.

Soru: 1 baytlık bir veriyi 4 baytlık bir alana aktarırsam veri kaybı yaşar mıyım? **Cevap:** Hayır. Doğrudan o alana saklanır. Hoca bunu 4 baytlık ve 1 baytlık iki kutu çizerek gösterdi.

Tersini düşündüğünüzde iş sıkıntılı olur. Hoca `double`'dan `int`'e dönüşümü denedi:

```
int t = (int)z;
```

Başka dillerde bunda sıkıntı yaşanacağını söyleyip Java'yı açtı ve aynısını orada denedi. Hata bekliyordu, ama derleyici hata vermedi. Sonra `t`'nin içindeki değere baktı: `t` 2 çıktı, 0.3'lük kısım atıldı. Buna kesme (truncation) denir. Tam kısım alınır, kalanı atılır.

> **Ders sonrası not:** Hoca bu yazımda Java'nın hata vermesini bekledi; doğrusu açık cast Java'da hata vermez. Bu makinede `java` 25 ile denendi: `int t = (int)z;` ve `z = 2.3` için çıktı 2, cast yazılmayan `int t = z;` için `incompatible types: possible lossy conversion from double to int` derleme hatası. Aynı C satırı gcc ile `z = 3.3` için 3 verdi.

Bu kayıp sizin için sorun olabilir. Hocanın örneği sayısal alandaki bir hesaptı. Virgülden sonraki üç basamağın önemli olduğu bir hesapta bu kısmı atarsanız sonuç yanlış gelir. Sonuç yanlışsa ulaşmak istediğiniz noktaya gidemezsiniz.

> **Hocanın vurgusu:** Dönüşümlerde dikkat etmemiz gereken şeyler genellikle örtük ifadelerdir.

7. Virgül Operatörü

Virgül operatörünün soldan sağa (left to right) birleşme özelliği vardır. Virgül koyduğunuzda önce soldaki, sonra sağdaki yapılır. Hoca önce şunu denedi:

```
double y = 2.3, yy = y + 1.0;
```

Değişken adları çakışınca derleyici yeniden tanımlama (redefinition) hatası verdi. Hoca adları değiştirip ikisini `%lf` ile yazdırdı ve 2.3 ile 3.3 gördü. Yorumu şuydu: Virgül soldan sağa çalışmasaydı, önce sağdaki yapılacaktı ve o değişken henüz olmadığı için hata alınacaktı.

> **Ders sonrası not:** Hoca bu satırı virgül operatörü örneği olarak verdi; doğrusu büyük olasılıkla buradaki virgül operatör değil, bildirim ayırıcısıdır. Sonuç yine aynıdır: gcc ile denendi, 2.300000 ve 3.300000 yazdı. Gerçek virgül operatörü örneği aşağıdaki `return` satırıdır.

Hocanın asıl uyarısı şuydu: Bazen öğrenciler ondalık sayıyı unutup virgülle yazıyor. `2,8` matematikteki iki tam onda sekiz değildir. C'de bu, 2 ve 8 sayıları demektir. Sayıyı kastediyorsanız `2.8` yazmalısınız. C burada da hata vermez.

Hoca bunu bir fonksiyonla gösterdi. Fonksiyona önce `f` adını verdi, ama aynı adda bir değişken olduğu için çakıştı. Fonksiyon ana programdan sonra yazıldığı için prototipini de yukarı ekledi.

```
#include <stdio.h>

int funct();

int main(void) {
    printf("%d\n", funct());
    return 0;
}

int funct() {
    return 2, 8, 7;
}
```

Soru: `return 2,8;` yazılırsa ekranda ne görürüz? **Cevap:** Hoca önce 2'nin görüleceği fikrini dile getirdi, sonra "acaba önce 2 mi göreceğiz?" diye sorguladı. Çalıştırınca 2 de 2.8 de görülmedi, en sağdaki değer olan 8 görüldü. `return 2,8,7;` yazınca da 7 görüldü.

Hoca, insanın aklına "önce 2'yi döndürür" fikrinin geldiğini, ama böyle olmadığını söyledi. Program 2, 8 ve 7'ye sırayla bakar, en son değeri döndürür. Hoca bunu stack mantığıyla da açıkladı: Stack'in tepesinde en son 7 kalıyor.

> **Ders sonrası not:** Hoca bunu stack mantığıyla açıkladı; doğrusu büyük olasılıkla şudur: Virgül operatörü soldaki ifadeyi hesaplayıp değerini atar, ifadenin değeri sağdaki ifadedir. Stack ile ilgisi yoktur. gcc ile denendi: Program 7 yazdı, `-Wall` iki kez `left-hand operand of comma expression has no effect` uyarısı verdi. Uyarıları açık tutmak bu hatayı yakalar.
>
> **Hocanın vurgusu:** Ondalık sayı kullanırken virgül ile noktaya dikkat etmemiz gerekir.

8. Arttırma Operatörleri

++ dikkatli kullanılması gereken operatörlerdendir. Hoca slayttaki şu kod üzerinden gitti:

```
int x = 3, y, z;
x++;            /* x = 4 */
++x;            /* x = 5 */
y = ++x + 3;
z = x++ + 3;
```

İlk iki satır sorun yaratmaz. x 3 ise bir arttırınca 4, sonra 5 olur. Sonek biçiminde (x++) arttırma ifadenin sonunda yapılır. Önek biçiminde (++x) ise doğrudan arttırılır.

Kafa karıştıran kullanım, ++'nın başka operatörlerle birlikte geldiği yerdir. O zaman şu tabloya bakılır: Önek arttırma (pre-increment) yukarıda, normal toplama daha aşağıda, atama en alttadır.

`y = ++x + 3;` satırı adım adım şöyledir:

1. Başlangıç: x 5.
2. Önce önek arttırma: x 6.
3. Sonra toplama: $6 + 3 = 9$.
4. En son atama: y 9.

`z = x++ + 3;` satırında hoca "önce eşittir operatörü gerçekleşiyor, yani x'i 3 ile topluyoruz, toplama arttırmadan önce oluyor" dedi.

> **Ders sonrası not:** Hoca bu satırda önce eşittirin çalıştığını söyledi; doğrusu büyük olasılıkla x++ tablonun en üstündedir ve önce işlenir, ama ifadeye x'in eski değerini verir. Artmış değer sonra yazılır. Sonuç olarak toplama eski değerle yapılır. gcc ile denendi: Yukarıdaki kodun sonunda y 9, z 9, x 7 çıktı.

Hocanın önerisi, bu ifadelere çok takılmamak için onları düzgün yazmak, gerekirse parantez kullanmak ya da tabloya bakıp hareket etmektir.

9. İlişkisel ve Mantıksal Operatörler

İlişkisel operatörlerin sonucu doğru ya da yanlıştır. Ama C'de `true` ve `false` sabitleri olmadığı için yanlışa karşılık 0, doğruya karşılık 1 üretilir.

Hoca bunu `if` bloğuyla anlattı. Bir `t` değişkeni olsun ve `t > 0` diye karşılaştıralım. Bunun başka ihtimali yoktur: Ya doğrudur, değeri 1 olur; ya yanlıştır, değeri 0 olur. Bu değere göre program akışı çeşitlendirilir: doğruysa şu kısmı yap, yanlışsa şu kısmı yap.

Birden fazla şart kullanacaksak VE, VEYA ve DEĞİL işlemlerine bakarız. Hoca bunları kümelerle ilişkilendirdi.

- **VE (&&):** İki kümenin kesişimi gibidir: hem A'da hem B'de olan durumlar. İki şartın da doğru olması gerekir. Herhangi biri 0 ise sonuç 0'dır.
- **VEYA (||):** İki kümenin birleşimi gibidir. Herhangi biri doğruysa sonuç doğrudur.
- **DEĞİL (!):** Doğruyu yanlışa, yanlışı doğruya çevirir.

Slaytta `x = 3`, `y = 0` örneği vardı:

- `x && y`: x 3 olduğu için doğru sayılır, yani 1 gibi görünür. y 0'dır. Sonuç 0, yani yanlış.
- `x || y`: x doğru olduğu için diğerine bakmaya gerek yoktur. Sonuç doğrudan 1'dir.

`x || y` için ikinci tarafa bakılmamasının nedeni, baştakinin 1 olmasının sonucu zaten doğru yapmasıdır. gcc ile denendi: `x && y`, `x || y`, `!x`, `!y` için 0, 1, 0, 1 çıktı.

Hoca bit düzeyindeki operatörlerin tek karakterli yazıldığını ve bit seviyesinde işlem yaptığını söyledi, burada anlatmadı.

10. printf ile Biçimlendirme

`printf` ekrana yazdırmak için kullanılır. İçinde bir string oluştururuz ve bu string içinde farklı veri biçimlerini belirtiriz.

> **Ders sonrası not:** Hoca "standart input'a veri gönderiyor" dedi; doğrusu büyük olasılıkla standart çıkıştır (stdout).

Tip	Belirteç
tam sayı	`%d` veya `%i`
`float`	`%f`
`double`	`%lf`
karakter	`%c`
işaretsiz	`%u`
string	`%s`

String tipi doğrudan yoktur, ama yazım biçimi `%s` olarak vardır. `%c` ve `%s` farklı şeylerdir.

10.1 Genişlik ve Ondalık Basamak

Virgüllü bir sayıyı yazdırırken `%N1.N2f` biçimi kullanılabilir. Hoca bunu virgülden önce ve sonra ne kadar yazılacağını söylemek olarak anlattı ve `%2.3f` ile dört basamaklı tam kısmı olan bir sayı denedi. Virgülden önceki 2 dikkate alınmadı, çünkü tam kısım iki basamaktan fazlaydı. Virgülden sonra üç basamak göründü. Sonra 2 yerine 8 yazdı, sonra 5 ile denedi. En sonunda başına bir 0 koyarak önünü sıfırla doldurmayı (zero padding) gösterdi. Bunun `double` ve diğer tipler için de yapılabileceğini söyledi.

> **Ders sonrası not:** Hoca N1'i virgülden önceki basamak sayısı gibi anlattı ve 8 yazınca "önüne sıfırlar koyuyor" dedi; doğrusu N1 alanın toplam en küçük genişliğidir, nokta ve ondalık basamaklar dahildir, doldurma da varsayılan olarak boşlukla yapılır. gcc ile denendi: `%2.3f` ve `%8.3f` ikisi de `4524.366`, `%10.3f` iki boşluk ve `4524.366`, `%010.3f` ise `004524.366` yazdı. Sayı alana sığmazsa kesilmiyor.

10.2 Kaçış Karakterleri

Hoca `printf`'in özel karakterlerini hatırlattı: yeni satıra geçmek için `\n`, boşluk bırakmak için sekme `\t`, dikey sekme `\v`, yeni sayfaya geçme ve imleci satırın başına getiren `\r`. Dikey sekmeyi denedi, ama kendi ortamında beklediği görüntüyü alamadı. Bunun her ortamda çalışmadığını söyledi.

11. Tam Sayı Bölmesi

11.1 Kesme, Yuvarlama Değil

İki tam sayı bölününce sonuç tam sayıdır. Hoca bunu öğrencilere sorarak ilerledi:

İşlem	Gerçek değer	C sonucu
`16/2`	8	8
`16/3`	5.33…	5
`15/2`	7.5	7

Soru: `16/3` için 5.33 gibi bir sonuç gelir mi? **Cevap:** Hayır, 5 gelir, çünkü ikisi de tam sayıdır.

Soru: `15/2` 7.5'tan 8'e yuvarlanır mı? **Cevap:** Hayır, 7 gelir. Hoca `16/3` örneğine dikkat çekti: 5.33 zaten 5.5'a varmadığı için "yuvarlamıyor" diye düşünebilirsiniz. Ama `15/2`'de de 8 değil 7 görürsünüz. Tam sayı bölmesinde yuvarlama yoktur, doğrudan atma vardır. Hepsi gcc ile denendi: 8, 5, 7.

11.2 Ondalıklı Sonuç Almak

Operandlardan biri ondalıklıysa sonuç ondalıklı olur. Hoca önce `%d` ile `15/2` yazdırdı ve 7 gördü. Sonra `15/2.0` yaptı, ama `%d` uygun olmadığı için sonuç bozuldu. Belirteci `%lf` yapınca 7.5 göründü. `15.0/2` de aynı sonucu verir.

Bazen şöyle yazılır: `(double)15/2`. **Soru:** Burada önce cast mı, bölme mi çalışır? **Cevap:** Bunu operatör önceliği söyler. Cast her zaman toplama, çıkarma, çarpma ve bölmeden önce gelir. Önce 15, 15.0 olur, sonra 2'ye bölünür ve 7.5 elde edilir.

Hoca sonra bir yazım daha denedi. Ekranda 0.0 gibi bir değer çıktı. Açıklaması şuydu: `15/2` tam sayı olarak 7'dir, değer olarak 7 vardır ama bu 7'nin bit gösterimi ondalıklı biçimde okununca 0 gibi yazılıyor. `float` için de aynısını gördü. Çözüm olarak önce bir değişkene kaydetmeyi önerdi:

```
float f2 = 15 / 2;
printf("%f\n", f2);   /* 7.000000 */
```

> **Ders sonrası not:** gcc ile denendi. `printf("%lf\n", 15/2);` satırı `-Wall` altında `format '%lf' expects argument of type 'double', but argument 2 has type 'int'` uyarısı verdi ve bu makinede 0.000000 yazdı. Bu, derste görülen 0.0 ile aynı. `(double)(15/2)` ise `%.1f` ile 7.0 yazdı: Önce parantezdeki tam sayı bölmesi yapılır, sonra 7, 7.0'a dönüşür. Ondalık kısım zaten kaybolmuştur.

11.3 Slayt Örneği

Slaytta bölme ve biçimlendirmeyi birleştiren bir program vardı. `a` 15 ve `b` için 15'in 2'ye bölümü alınıyordu.

```
#include <stdio.h>

int main(void) {
    int a = 15, b;
    float c = 15.3, d;

    b = a / 2;
    printf("%d\n", b);    /* 7   */
    printf("%3d\n", b);   /* "  7" */
    printf("%03d\n", b);  /* 007 */

    d = c / 3;
    printf("%3.2f\n", d); /* 5.10 */
    return 0;
}
```

- `%d`: 7.
- `%3d`: 3 karakterlik yer ayırır, sayıyı sağa yaslar. Önüne iki boşluk gelir.
- `%03d`: Önünü sıfırla doldurur: 007.
- `%3.2f`: `c` ondalıklı, 3 tam sayı olduğu için sonuç otomatik olarak ondalıklı olur. $15.3/3 = 5.1$. Virgülden sonra 2 basamak yazılır: 5.10.

Hoca `%13d` yazınca işin karıştığını söyledi. Burada doldurma karakteri olarak yalnız sıfır verilebilir. Bir verilseydi, o 1 ile yanındaki 3 birbirine karışırdı, 13 sayısı olarak okunurdu. Bu yüzden yalnız 0 istisna tutulmuş. gcc ile denendi: `%13d` 12 boşluk ve 7 yazdı, yani 13 bir genişliktir.

> **Ders sonrası not:** Hoca `%3.2f` için "3 basamağı yazacak şekilde tam kısmı oluşturuyor" dedi; doğrusu 3 toplam en küçük genişliktir. `5.10` dört karakter olduğu için

3 burada etkisizdir. gcc ile denendi, çıktı `5.10`.

12. Atama: L-value ve R-value

Programlamayı ilk kez görenlere atama biraz karışık gelebilir. Atama, sol değer (L-value) ve sağ değer (R-value) arasında yapılan bir işlemdir. Daima sağ taraftaki değer sol tarafa kopyalanır.

İki taraf arasında bir denge kurmak gerekir:

- Sağ tarafta da sol tarafta da değişken adı yazılabilir.
- Sağ tarafta operatör kullanılabilir, ama sol tarafta kullanılamaz. Bu yanlıştır.
- Sağ tarafta `num * 2` ya da `num + 5` gibi şeyler yazılabilir. Ama `5 = answer` gibi bir şey yazılamaz. gcc ile denendi: `lvalue required as left operand of assignment` hatası verdi.

L-value yalnız değişken olabilir: dizi elemanları ya da normal değişkenler. R-value ise değişkenleri, sabit sayıları (literal: 3, 5, 10) ve pi sayısının tanımlanması gibi sabitleri (constant) içerir. Hoca bunu küme olarak anlattı: L-value, R-value'nun bir alt kümesi gibidir.

13. Kapsamlı Örnek: Dönüşümler ve İkiye Tümleyen

Aradan sonra hoca, cast operatörlerini ve o ana kadar görülenleri birleştiren ayrıntılı bir kod dosyasını inceledi. Kısa notta bu dosyanın adı `hafta2_impl_expl_cast_operators.c` olarak geçiyor.

13.1 Bölme ve Toplama Birlikte

Soru: `-3/4 + 2.5` ifadesinin sonucu nedir? **Cevap:** Bir öğrenci 1.75 dedi ve bölmenin öncelikli olduğunu düşündüğünü söyledi. Hoca bölmenin önce yapılacağında sorun olmadığını, ama tam sayı bölmesine dikkat edilmesi gerektiğini söyledi.

Adımlar şöyledir:

1. Bölme önce yapılır: `-3/4`. İkisi de tam sayıdır. Sonuç -0.75 değildir.
2. Virgülden sonraki kısım kesilir, sonuç 0 olur.
3. $0 + 2.5 = 2.5$. Tam sayı olan 0, ondalıklı 2.5 ile toplanır.

Hoca bunu `printf` ile gösterdi. `-3/4`, `%d` ile 0 yazdı. Paydayı `4.0` yapınca 0.75'li sonuç görüldü: gcc ile denendi, `%f` ile -0.750000.

`-3/4.0 + 2.5` ifadesinde `4.0`, bölmeyi otomatik olarak ondalıklı yapar. Sonra her şey ondalıklı devam eder: $-0.75 + 2.5 = 1.75$. Hoca bu sonucu "1.75 mi oluyor, neyse öyle bir şey" diye geçti. python ile hesaplandı, 1.75 doğrudur.

Soru: `1 + 2.5` kaç eder? **Cevap:** 3.5. Küçük tip her zaman büyük tipe dönüşür: `int`, `float`'a ya da `double`'a dönüşür.

> **Ders sonrası not:** Hoca aynı yerde "string'in sonuna integer eklerseniz o da string olur" dedi; doğrusu büyük olasılıkla bu C için geçerli değildir, Java gibi dillerin özelliğidir. C'de string tipi zaten yoktur (Bölüm 2.1).

13.2 unsigned ile Çıkarma

Kodda `10u` gibi bir ifade vardı. Sonundaki `u` işaretsiz (unsigned) demektir. Hoca önce `10u` ve `10`'u `%u` ile `%d` ile yazdırdı: ikisi de 10. Sonra 15 çıkardı. `%d` ile -5 görüldü. `%u` ile yazdırınca çok büyük bir sayı çıktı. gcc ile denendi: `printf("%u %d\n", 10u-15, 10u-15);` satırı `4294967291 -5` yazdı.

Hocanın açıklaması, aynı bit dizisinin farklı okunmasıdır. Bunu 3 bitlik bir tabloyla gösterdi. İşaretsiz okuduğumuzda:

Bit dizisi	İşaretsiz değer
000	0
001	1
010	2
011	3
100	4
101	5
110	6
111	7

Örneğin 110 işaretsiz okununca 6'dır. Aynı diziyi işaretli kabul edersek bir bit işarete gider ve dizi -2 olur. Yani dizi aynı, değer farklı. Hoca 101'in -3'e, 110'un -2'ye karşılık geldiğini de bir hesaplayıcıda gösterdi. Bu değerler python ile hesaplandı.

Aynı şey 32 bitte de olur. $10 - 15 = -5$'tir. Bu -5'in bit dizisini işaretsiz okursak çok büyük bir sayı görürüz.

13.3 -5'in İkiye Tümleyeni

Hoca önce -5'i işaret biti 1, sonda 101, arası sıfırlarla yazdı. Sonra bu konunun BBG dersinde görülüp görülmediğini sordu. Öğrenciler ikiye tümleyen (two's complement) sayıları gördüklerini söyledi. Hoca bir hesaplayıcıda -5'in gösterimini açtı.

Soru: Siz oraya sıfırlar koydunuz, bir mi koyacağız? **Cevap:** Evet, orada birler olacak. Hoca 5 yazdığı yerde 5'in düz gösterimini koyduğunu, ikiye tümleyende hesabın farklı olduğunu söyledi.

İkiye tümleyen şöyle bulunur. Hoca önce "1 ekliyorduk" dedi, hemen düzeltti: Önce devrik alınır, sonra 1 eklenir.

1. 5'in ikilik gösterimi: 00000000 00000000 00000000 00000101.
2. Bitleri ters çevir: 11111111 11111111 11111111 11111010.
3. 1 ekle: 11111111 11111111 11111111 11111011.

Hocanın kontrolü şuydu: Sağdan üçüncü basamak 0, geri kalan her şey 1, işaret biti de 1. Bu dizi -5'tir. Aynı diziyi işaretsiz düşünürseniz o büyük değeri bulursunuz. python ile hesaplandı: 11111111111111111111111111111011 işaretsiz olarak 4294967291'dir, yukarıdaki gcc çıktısıyla aynıdır.

Hoca sonucu çevrim içi bir ikilik-onluk çevirici ile de göstermek istedi. Ama çevirici 20 basamak aldığı için bu mümkün olmadı.

13.4 Sınıfın Seçtiği Örnek: -86

Hoca öğrencilerden bir sayı istedi, bir öğrenci -86 dedi.

Önce 86'nın ikilik açılımı bulunur. Hoca basamak değerlerini saydı: 1, 2, 4, 8, 16, 32, 64.

1. 86 içinde 64 vardır: $86 - 64 = 22$. 64'ler basamağı 1.
2. 22 içinde 16 vardır: $22 - 16 = 6$. 16'lar basamağı 1. 32'ler basamağı 0.
3. 6 için 4'ler ve 2'ler basamağı 1, 8'ler ve 1'ler basamağı 0.
4. Kontrol: $64 + 16 + 4 + 2 = 86$.

Hoca ilk yazımda bir basamağı yanlış koyup kontrol ederken fark etti ve düzeltti. Doğru dizi 1010110'dur. Öndeki bitlerin hepsi 0'dır.

Sonra ikiye tümleyen:

```
86           : 00000000 00000000 00000000 01010110
birler tüm.  : 11111111 11111111 11111111 10101001
1 eklenince  : 11111111 11111111 11111111 10101010
```

Toplamayı hoca sağdan yaptı: 1 + 1 = 2, yani 0 yaz, elde 1. Elde bir sonraki basamaktaki 0'a eklenir ve 1 olur. Geri kalan bitler değişmez. Birler tümleyeni (one's complement), bütün

sıfırları bir ve bütün birleri sıfır yapmaktır. Buna 1 eklenince ikiye tümleyen bulunur. Hesaplayıcıyla karşılaştırınca sonucun doğru çıktığı görüldü: son bayt dört tane `10` çiftidir, kalan her şey 1'dir. Değerler python ile hesaplandı ve aynı çıktı.

Hoca, bu diziyi işaretsiz okuyunca çok büyük bir sayı çıktığını söyledi: $2^{31} + 2^{30} + 2^{29} + \ldots$ biçiminde birli bitlerin kuvvetleri toplanır. python ile hesaplandı: Toplam 4294967210'dur.

> **Ders sonrası not:** Hoca bu noktada "unsigned" ile "signed" sözcüklerini karıştırdı ("sinned bir sayı gibi okursak çok büyük"); doğrusu bu dizi işaretli okununca -86, işaretsiz okununca 4294967210'dur (python ile hesaplandı). Hocanın "31 tane basamaklı" dediği dizi de 32 bittir.

13.5 Atamada Kesme

Kodda şu satırlar vardı:

```
float f = 15;
int j = 5;
printf("%2.2f\n", j + f + 2.5);
```

15 bir tam sayıdır, ama `float`'a atanınca otomatik olarak 15.0 kabul edilir.

Soru: Bu `printf` ne yazar? **Cevap:** Bir öğrenci 22.5 dedi. Hoca sayının doğru olduğunu, ama biçimin de önemli olduğunu söyledi: $5+15+2.5 = 22.5$. `%2.2f` ile virgülden sonra iki basamak yazılır, buçuğun yanına bir sıfır gelir: 22.50. gcc ile denendi, 22.50 yazdı.

> **Hocanın vurgusu:** Bunlar önemli. Bu tarz şeyler sınavlarda sorulabiliyor. Hem operatör önceliğini hem değişkenlerin durumunu ölçer.

Sonraki satır şuydu:

```
j = j + f + 2.5;
printf("%d\n", j);
```

Soru: Eşitliğin sağ tarafı `float` mı `int` mi? **Cevap:** Bir öğrenci `float` dedi. Hoca onayladı: İfadede bir `float` varsa diğerleri de ona dönüşür. Biri `double` olsaydı hepsi `double` olurdu.

Adımlar:

1. Önce `j + f`: $5 + 15.0 = 20.0$.
2. Sonra `+ 2.5`: 22.5.
3. Atama: `j` bir `int` olduğu için 22.5 tam sayıya dönüşür.
4. Yuvarlama yapılmaz, kesme yapılır. Sonuç 23 değil 22'dir.

Yuvarlama yapsaydık 23 gibi bir şey görürdük. Virgülden sonrası atılır.

> **Ders sonrası not:** Hoca buradaki `2.5` için "bunlar bir float" dedi; doğrusu büyük olasılıkla sonunda `f` olmayan `2.5` sabiti `double`'dır, dolayısıyla `j + f` `float`, sonraki toplama `double` olarak yapılır. Bu sonuç olarak 22'yi değiştirmiyor. gcc ile denendi: 22.

Sonra `j = 2.5;` için doğrudan 2, `j = 999.888888;` için doğrudan 999 görülür. Bir öğrencinin başka bir yazımla ilgili sorusuna hoca yine "doğrudan atıyorsunuz, yalnız tam kısmı kalır" diye cevap verdi. gcc ile denendi: 2 ve 999.

13.6 INT_MAX Neden Bir Eksik?

Kodda `INT_MAX` da geçiyordu. Değeri $2^{31} - 1$'dir. Hoca nedeni sordu: Neden tam 2^{31} değil? Çünkü aralığın içinde 0 da vardır ve 0, işaretli sayılarda pozitif tarafa dahildir.

Bu, ikiye tümleyen gösteriminden gelir. Birler tümleyeninde $+0$ ve -0 diye iki sıfır tanımlıdır. İkiye tümleyende bu ortadan kalkar. Hoca bunu 3 bit üzerinde gösterdi:

- $+1$ `001`'dir.
- -1 için bitleri ters çevir: `110`. 1 ekle: `111`.

Sıfırın gösterimi `000`'dır ve ikiye tümleyende tek sıfır vardır. Bu sıfırın işaret biti 0 olduğu için pozitif tarafa düşer. Pozitif taraf 2^{31}'den bir önceki sayıda biter, çünkü bir sonrakine yer kalmaz. gcc ile denendi: `INT_MAX` 2147483647, `INT_MIN` -2147483648 yazdı.

13.7 Açık Dönüşüm: `(float)k/j`

Kodda bir de `k` 3, `j` 2 iken `(float)k/j` vardı. `k`'yi `float`'a çevirmek, 3 olan `k`'yi 3.0 yapmak demektir. 3.0, 2'ye bölününce 1.5 elde edilir. `f` de `float` olduğu için 1.5 görülür.

> **Ders sonrası not:** Aynı kodda cast olmadan `f = k/j;` yazılırsa ne olacağı altyazıda söylenmiyor, kısa notta 1.0 yazıyor. gcc ile denendi: 1.000000 ve 1.500000 yazdı.

Hoca kodun devamındaki satırları tek tek anlatmayacağını, dosyanın paylaşıldığını, inceleyip anlaşılmayan yerleri sormamızı söyledi.

14. printf İçinde Arttırma ve m++ * m++ Tartışması

14.1 printf Argümanlarında Arttırma

Hoca `sizeof` için önceki anlatımı hatırlatıp geçti. Sonra `j` 5 ve `k` 5 iken küçük bir örnek gösterdi. Önekte, atama işlemi yapılmadan önce arttırma yapılır:

```c
#include <stdio.h>

int main(void) {
    int j = 5, k = 5;
    printf("%d %d\n", ++j, --k);   /* 6 4 */
    printf("%d %d\n", j, k);       /* 6 4 */

    j = 5;
    k = 5;
    printf("%d %d\n", j++, k--);   /* 5 5 */
    printf("%d %d\n", j, k);       /* 6 4 */

    j = 5;
    printf("%d\n", j++);           /* 5 */
    printf("%d\n", ++j);           /* 7 */
    return 0;
}
```

İlk `printf` önce değerleri değiştirir: 6 ve 4. Sonraki satır aynı güncel değerleri gösterir. Sonekte ise değer önce yazılır, sonra güncellenir. Hoca ikinci durumda 5 ve 5 beklediğini söyledi. Güncel değerleri bir sonraki satırda görürüz. Son olarak `j` 5 iken önce sonek, sonra önek kullanılırsa 5 görülür, `j` 6 olur, sonraki satırda da 7 görülür. Bu kod gcc ile denendi, yorumlardaki değerleri yazdı.

14.2 `n = m++ * m++`

Sonraki örnek daha karışıktı. `m` 1 iken `n = m++ * m++;` satırında ne olur? Hoca bunun kendi kafasını da karıştırdığını, algoritma ve programlamayla çok ilgisi olmadığını söyledi.

Hocanın beklentisi operatör tablosundan geliyordu:

1. Sonek arttırma tablonun en üstündedir, çarpma ondan aşağıdadır.
2. En son eşittir çalışır.
3. O zaman ilk `m++` ile 2 olur, ikinci `m++` ile 3 olur.
4. $2 \times 3 = 6$, `n` 6 olur, `m` 3 olur.

Çalıştırdığında ise `n` 2 çıktı, `m` 3 çıktı. Hoca bunun tablonun biraz dışına çıktığını söyledi. Derleyicinin kendi algoritmaları olabileceğini, araştırıp farklı derleyicilerde karşılaştıracağını belirtti.

Bir öğrenci çevrim içi bir derleyicide (programiz) denediğini, n için 2, m için 3 gördüğünü söyledi. Hoca bir başka çevrim içi derleyicide (GDB) de denedi.

Soru: Sonekte önce `m` ile işlem yapılıp sonra arttırılmıyor muydu? O zaman 2 ve 3 mantıklı değil mi? **Cevap:** Hoca bunu birlikte düşündü. Öğrencinin açıklaması şuydu: İlk `m` 1 değeriyle çarpıma girer, sonra 2 olur. İkinci `m` 2 değeriyle girer, sonra 3 olur. 1×2, `n`'e atanır. Hoca, kendi yorumunun tabloya göre 2×3 olduğunu, ama sonucun bu açıklamayla uyuştuğunu kabul etti: "Yani burada önce 1 var, sonra `m` 2 olduğu için burada 2 oluyor, `m`'nin kendisi de 3 oluyor."

Soru: Artıları sola eklesek ne olur? **Cevap:** Öğrenciler 6 beklediklerini söyledi. Hoca denedi ve 9 gördü. "Bir dakika, bundan niye 9 oldu?" diye sordu. Bu soru derste açık kaldı.

Soru: ++ operatörü satırın tamamını alıyor olabilir mi? Yani sondaki ++, çarpmanın da yapılmasını bekliyor olabilir mi? **Cevap:** Hoca bunu operatör önceliği açısından tekrar inceleyeceğini, kullandıkları kitaptan alınan bir örnek olduğunu, açıklaması varsa paylaşacağını söyledi.

Hoca birkaç deneme daha yaptı. `printf` içinde `m++` yazdırmayı denedi ve bu tür ifadelerin biraz aldatıcı (tricky) olabildiğini, normalde durduk yere karşımıza çıkmadığını söyledi.

> **Hocanın vurgusu:** Bunlar çok tercih edilen, algoritma ve programlama açısından çok lazım olan şeyler değil. Sınavda sorsak da bu tarz doğrudan kafa karıştırıcı şeyler sormuyoruz.

15. İkili Mantık Tekrarı

Hoca sonra ikili mantık (binary logic) ve önermeler mantığının temel kısımlarına baktı. C'de Boolean diye bir değişken türü yoktur. Başka dillere hâkim olanlar bunu farklı bilebilir. Kütüphanesi eklenerek sağlanabilir, ama bu derste onunla işimiz yok. Doğru 1, yanlış 0'dır.

> **Ek bilgi (derste anlatılmadı):** Hocanın bahsettiği kütüphane büyük olasılıkla C99 ile gelen `<stdbool.h>` başlığıdır. `bool`, `true` ve `false` adlarını sağlar.

VE işleminin özelliği şudur: İşlemin üzerinde çalıştığı değerlere operand denir, işlemin kendisi operatördür. İki operand da 1 ise sonuç 1, diğer durumlarda 0'dır.

Hocanın anlam farkı örneği şuydu:

- "Öğrencinin devamsızlığı 10 saatten az **ve** ortalaması 65'ten fazla olacak, ancak o zaman geçer": İki şartın ikisi de sağlanmalıdır.
- "Devamsızlığı 10'dan küçükse **veya** ortalaması 65'ten büyükse geçebilir": Devamsızlığı 10'dan küçük bir kişi, ortalaması 0 olsa da geçebilir. Anlamsız da olsa mantık bunu söyler.

Tip dönüşümlerinin ve ikilik-onluk dönüşümlerin ayrıntısına girmedi. Eksiği olanların slaytlardan çalışmasını, ikilikten onluğa ve onluktan ikiliğe dönüşümün ileride lazım olacağını söyledi. Kaydırma (shift) gibi bit operatörlerini daha sonra konuşacaklarını belirtti.

16. m++ * m++ İfadesine Assembly ile Bakmak

Bu sırada hocaya yeni bir sonuç geldi. Hoca üç çarpanlı bir denemeyi konuştu: "1 çarpı 2 çarpı 3 olur, o zaman 6 olur." Bundan, derleyicinin mevcut değeri çarpma için tutup `m`'nin bir sonraki değerine geçtiği sonucunu çıkardı.

Hoca "madem girdik, biraz üzerinde duralım" diyerek kodu, derleyicinin assembly çıktısını gösteren çevrim içi bir sitede açtı ve 64 bit gcc çıktısını okudu.

Hocanın okuduğu akış şuydu:

1. Bellekten `m` getirilir ve EDX yazmacına kaydedilir. Bu değer 1'dir.
2. EAX üzerinden değer bir arttırılır. Bellekteki `m` 2 olur. EDX'teki kopya hâlâ 1'dir.
3. Bellekten `m` tekrar getirilir. Bu kez 2 gelir.
4. Bu da bir arttırılır. Bellekteki `m` 3 olur.
5. İki değer çarpılır: EAX ile EDX, yani $2 \times 1 = 2$.
6. Sonuç `n`'e yazılır: `n` 2, `m` 3.

Yani iş bitmeden önceki eski değer yazmaçta tutuluyor. Hoca bunun iki adımda çalıştığı için böyle olabileceğini, başka bilgisayar mimarilerinde de denemek gerektiğini söyledi. Assembly tarafına girilirse işin içinden çıkılamayacağını, derleyici açısından yeniden inceleyip döneceğini belirtti.

> **Ders sonrası not:** Hoca cevabı araştırıp döneceğini söyledi ve operatör önceliğine göre ++'nın önce yapılması gerektiğini düşündü; doğrusu büyük olasılıkla şudur: Öncelik yalnız hangi operandın hangi operatöre ait olduğunu belirler, hesaplama sırasını belirlemez. Aynı değişkeni bir ifadede iki kez değiştirmek C standardında büyük olasılıkla tanımsız davranıştır. gcc ile denendi: `gcc -Wall` bu satır için `operation on 'm' may be undefined [-Wsequence-point]` uyarısı verdi.

Aynı denemelerin bu makinedeki gcc çıktısı şöyledir:

İfade (`m` başta 1, yerel değişken)	`n`	`m`
`n = m++ * m++;`	2	3
`n = ++m * ++m;`	9	3
`n = m++ * m++ * m++;`	6	4

İlk iki satır derstekiyle, üçüncüsü hocanın $1 \times 2 \times 3$ yorumuyla aynıdır, ama bunlar garanti edilen sonuçlar değildir. Önek örneğinde `m` global değişken yapılıp `-O0` ile derlenince `n` 9 yerine 6 çıktı. Yani aynı derleyicide, değişkenin nerede tanımlandığına göre bile sonuç değişebiliyor. Dersteki "bundan niye 9 oldu?" sorusunun cevabı da budur: Sonuca güvenilemez.

17. Numaralandırma (enum)

Diziler sonraki derse bırakıldı. Dersi numaralandırma tipleriyle (enumeration) bitirmek üzere hoca bu konuya geçti.

`enum` bir numaralandırma aracıdır. Haftanın günleri, yılın ayları gibi değişkenlerde string kontrolü yapmak zordur. Pazartesi'yi string olarak tutarsanız, bir yerde yanlış yazmanız string karşılaştırmasında sorun çıkarır. `enum` bu işi daha kolay ve sayısal değerler üzerinden yapmamızı sağlar. Güncel üst seviyeli dillerde bu daha kolaydır.

`enum` yazıp alabileceği değerleri sıralarsınız ve buna bir etiket verirsiniz. Değerler otomatik numaralanır:

```
enum gun { Mon, Tue, Wed, Thu, Fri, Sat, Sun };
```

Bir `today` değişkeniyle `today == Mon` gibi bir karşılaştırma yapılabilir. String üzerinde böyle bir karşılaştırma yapmak isteseydik, `==` operatörünün string için tanımlı olması gerekirdi. Ama stringler için böyle bir şey tanımlı değildir. `enum` bu dezavantajı ortadan kaldırır.

> **Ek bilgi (derste anlatılmadı):** C'de iki stringi karşılaştırmak için büyük olasılıkla `<string.h>` içindeki `strcmp` kullanılır. Karakter dizileri için `==` yazmak içerik değil adres karşılaştırır.

Tuesday atayınca otomatik olarak 1 değeri görülür. Saturday atayınca da onun değeri aktarılır. Görünürde etiketlerle çalışıyoruz, ama arka planda bu iş sayılarla yapılır. Derste çıktıda 1 ve 5 görüldü. gcc ile denendi: `Tue` 1, `Sat` 5.

Bir elemana ilk değer verirsek, bir sonraki ilk değeri görene kadar artı bir, artı bir diye devam edilir:

```
#include <stdio.h>

enum e { A = 3, B, C, D = -6, E, F = 10, G };

int main(void) {
    printf("%d %d %d %d %d %d %d\n", A, B, C, D, E, F, G);
    return 0;
}
```

3 atadıysak sonrakiler 4, 5 olur. Bir eleman -6 olduğu için sonraki 6 değil, -5 olur. 10 atanınca sonraki 11 olur. Çıktı: 3, 4, 5, -6, -5, 10, 11. gcc ile denendi ve aynı çıktı. Bu kod, kısa nottaki eleman adlarıyla yazıldı.

Bu tip, `if` içinde, daha çok `switch` ile işe yarar. Hoca `switch`'i henüz anlatmadığını, farklı durumlar (case) için bu değerlerin kullanılacağını söyledi.

18. Ömür (Duration): Stack ve Heap

Hoca geçen ders anlattığı ömür (duration) konusunu kısaca özetledi. Bir değişken tanımlandığı bloğun içinde vardır. Burada iki durum vardır:

- Değişken stack bölgesinde oluşturulduysa, blok bitince o değişken ölür.
- Değişken heap bölgesinde oluşturulduysa, blok bitse bile yaşamaya devam eder.

Soru: Normal bir `int x` hangi bölgededir? **Cevap:** Stack bölgesinde. Sabit uzunluklu diziler de stack bölgesindedir. `malloc` ile oluşturulanlar heap bölgesindedir.

Bunlar `main` içindeyse sorun olmaz, çünkü program kapanana kadar yaşarlar. Asıl mesele alt fonksiyonlardadır:

```c
#include <stdlib.h>

void alt_fonksiyon(void) {
    int x;                                  /* stack */
    int dizi[5];                            /* stack */
    int *dizi2 = malloc(5 * sizeof(int));   /* heap  */
    /* ... */
}
```

Fonksiyon tamamlandıktan sonra stack'teki değişkenlerin yerlerinde artık söz hakkınız yoktur. O yeri başka bir alt program kullanabilir. Heap'te oluşturulan yer ise fonksiyon sonlansa bile, siz `free` etmedikçe yaşamaya devam eder. Yukarıdaki `dizi2` için açılan 5 elemanlık yer, program kapanana kadar ya da siz `free` edene kadar vardır. `free` etmezseniz bellek sızıntısı (memory leak) sorunuyla karşılaşırsınız.

> **Ders sonrası not:** Hoca heap'teki bellek için "static duration veya fixed duration" dedi, başka bir yerde "dynamic duration" da dedi; doğrusu büyük olasılıkla dinamik (allocated) ömürdür. Statik ömür, global ve `static` değişkenler içindir, stack'teki yerel değişkenler ise otomatik ömürlüdür.

Hoca bu kavramın `malloc` ve fonksiyonlar daha ayrıntılı konuşulduğunda anlamlı hale geleceğini söyledi.

Dersin Toparlaması

Ders, Von Neumann mimarisi ve bellek hiyerarşisinin kısa tekrarıyla başladı ve programın bellekte nasıl durup çalıştığını hatırlattı. Sonra verinin bellekte nasıl tutulduğuna geçildi: `int` için 32 bit ve ikiye tümleyen, `float` ve `double` için IEEE 754, `char` için tek bayt ve ASCII. Aynı bit dizisinin işaretli ve işaretsiz okunduğunda farklı değerler vermesi, `10u - 15` örneğiyle ve 3 bitlik tabloyla gösterildi.

Dersin asıl konusu tip dönüşümü ve operatörlerdi. Örtük dönüşümlerin koddan görünmediği, tam sayı bölmesinde ve ondalıktan tam sayıya atamada yuvarlama değil kesme yapıldığı, operatör önceliğinin ifadeleri nasıl yönlendirdiği ve `printf` belirteçlerinin tipe uygun seçilmesi gerektiği örneklerle işlendi. `m++ * m++` tartışması açık kaldı ve hoca yanıtı araştırıp dönecek. Dersin sonunda `enum` ve stack ile heap arasındaki ömür farkı görüldü. Bir sonraki derste diziler incelenecek. Ömür konusu da `malloc` ve fonksiyonlarla birlikte ayrıntılanacak.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.